

**** Verification Academy News ****X
close**Upcoming Events**

- [VA Live - El Segundo \(https://www.eventbrite.com/e/verification-academy-live-el-segundo-tickets-529881789697\)](https://www.eventbrite.com/e/verification-academy-live-el-segundo-tickets-529881789697) | - March 7th
- [SystemVerilog Assertions \(https://event.on24.com/wcc/r/4065004/C3105EA4B3FE9E6D6BC8DB2AD19A176?partnerref=VA\)](https://event.on24.com/wcc/r/4065004/C3105EA4B3FE9E6D6BC8DB2AD19A176?partnerref=VA) | March 15th

UVM Framework

- [UVMF 2023.1 \(https://verificationacademy.com/news/uvm-framework-package-and-sessions-update\)](https://verificationacademy.com/news/uvm-framework-package-and-sessions-update) - New Release & Sessions

Verification Horizons

- [Verification Horizons \(https://verificationacademy.com/verification-horizons/march-2023-volume-19-issue-1/\)](https://verificationacademy.com/verification-horizons/march-2023-volume-19-issue-1/) - Read the March issue
- [Verification Horizons Blog \(https://blogs.sw.siemens.com/verificationhorizons/\)](https://blogs.sw.siemens.com/verificationhorizons/) - Read the latest posts

Home (/) > [Verification Horizons \(https://verificationacademy.com/verification-horizons/\)](https://verificationacademy.com/verification-horizons/) >
[March 2023 \(https://verificationacademy.com/verification-horizons/march-2023-volume-19-issue-1/\)](https://verificationacademy.com/verification-horizons/march-2023-volume-19-issue-1/) >
 Jumpstart your formal verification with a little help

Jumpstart your formal verification with a little help

[horizons/march-2023-volume-19-issue-1\)](https://verificationacademy.com/verification-horizons/march-2023-volume-19-issue-1/)

by Doug Smith - Doulos

An advantage of using formal verification is how quickly a formal environment can be created with a few simple properties that immediately start finding design issues. However, not all design behaviors are easily modeled using SystemVerilog's property syntax, resulting in complex or numerous properties, or behaviors that require more than just SVA. That is where helper code comes to the rescue. Helper code can significantly reduce the complexity of properties as well as be used to constrain formal analysis. Likewise, formal analysis may need to reduce the complexity of the problem and state space, which helper code can also help. So where are some places to use helper code and when? This article looks at how helper code can be used to simplify our properties, model formal abstractions, constrain formal inputs, and aid formal analysis.

I Need Help!

A formal tool is directed by SystemVerilog assertions (SVA). SVA properties provide both the targets for formal analysis and the constraints for the formal inputs. Ideally, our properties can take advantage of the SVA temporal syntax, which eliminates the needs for complicated state machines to model the design's behavior.

While most formal targets are written using SVA syntax, there are a number of scenarios where modeling design behavior may be more difficult to accomplish with SVA. In fact, not everything is allowed within an SVA property. For example,

- + Real numbers are not supported (some formal tools have partial support for reals such as Questa Formal)
- + Timing delays are not allowed
- + Asynchronous behaviors are difficult to model since SVA is synchronous by definition
- + Updating variables outside of properties is not allowed without the use of tasks and functions

Furthermore, some things are simply hard to model using SVA. Suppose an out-of-order bus protocol is being verified. Local variables within a named property can be used to capture transaction ids, but matching up the responses as they come back in an out-of-order fashion is challenging. Grabbing values from the past with an indeterminate lookback range is also difficult since `$past()` requires a constant lookback number of ticks.

Another challenging aspect of using SVA with formal is thoroughness. Since formal explores all possible scenarios, writing constraints for control logic is not always intuitive. Any scenario not fully described may result in invalid results. Thus, it is often easier to simply write a Verilog process to describe the behavior so formal knows exactly how the control logic behaves. Writing additional Verilog code, or *helper code*, can greatly simplify writing formal properties. What might be a complex SVA property becomes just a simple assertion.

Writing helper code is often how we create abstractions for formal analysis. It can be used for defining asynchronous controls in a property's *disable iff* statement. Further, many features of formal can only be described using formal directives like loading in the state space from a waveform file, cutting nets, or describing the reset sequence. In other words, while we typically focus on writing properties, understanding when and where to use helper code can save time and effort, and aid formal when targets struggle to converge. Therefore, this article presents a number of practical examples to illustrate how helper code can help you with your formal verification.

Startup Constraints

One challenge you may have encountered is how to create a constraint that only applies when starting up out of reset.

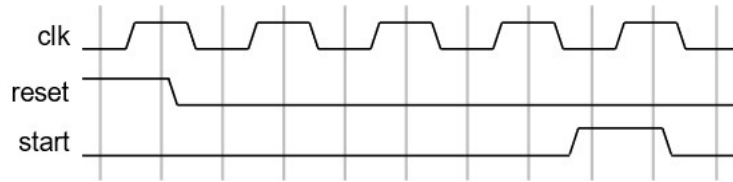


Figure 1 - Create a constraint that runs only once after reset.

Writing a simple property does not necessarily work because a property must hold true for the entire formal evaluation. For example, this property describes our waveform, but formal would only be able to meet this constraint the first four cycles coming out of reset and then things would no longer match:

```
property prop_init_seq;
  !start[*3] ##1 start
endproperty
```

The solution is to place the initialization sequence inside of an *initial block*, and formal will only evaluate the property *once* when reset deasserts:

```
initial begin : INIT_SEQ
  asm_init_seq: assume property ( prop_init_seq );
end
```

In this case, the Verilog initial block process acts as the helper code to tell formal when to evaluate the property and to only evaluate it once.

Back-to-Back Transactions

Another useful place for helper code is when dealing with multi-cycle bus protocols. Consider the simple APB bus protocol as an example. Both the read and the write cycles require 2 clock cycles to complete. The PSEL indicates that the cycle has begun, and the PENABLE indicates that the cycle is complete.

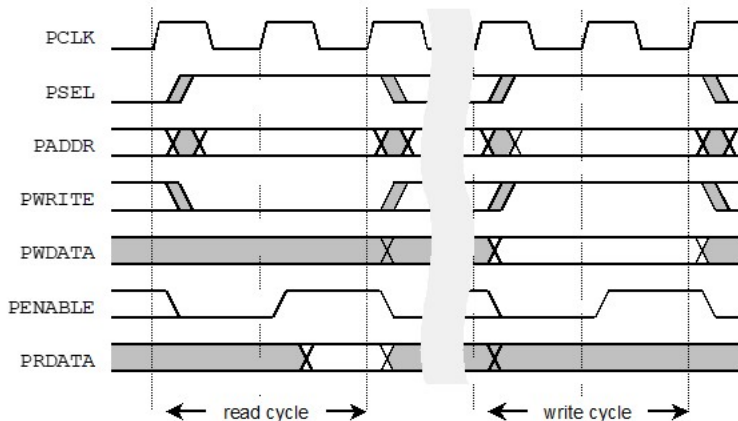


Figure 2 - Two cycle APB bus protocol.

This protocol is so simple that an assertion can easily be written for it:

```
assert property ( psel ==> penable );
```

The assertion works in this simple scenario, but fires in the case of back-to-back transactions where PSEL is held high more than two cycles:

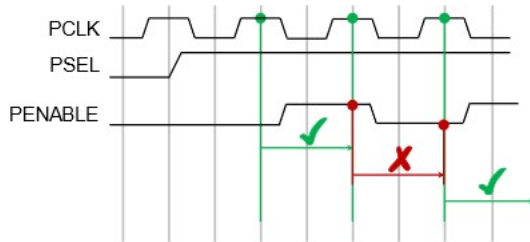


Figure 3 - Back-to-back APB transactions.

A possible workaround would be to anchor to the beginning of the transaction, but this would only match for the first transaction and all subsequent transfers would be ignored:

```
assert property ( $rose(psel) | => penable );    // Only checks once
```

An easy solution is to use a state variable to keep track of the beginning of the transaction:

```
int active = 0;
function void set_active(int state);
    active = state;
endfunction
property valid_apb_cycle;
    ( psel && !active, set_active(1) ) | =>
        ( penable, set_active(0) );
endproperty
ast_vld_cycle: assert property ( valid_apb_cycle );
```

The function `set_active()` is used by the named property to set or toggle the active flag. Alternately, this could be written using a process:

```
always @(posedge clk or negedge resetn)
    if ( !resetn ) active = 0;
    else
        if ( psel ) active ^= 1;
        else
            active = 0;
ast_vld_cycle: assert property ( psel && !active | => penable );
```

Using a Verilog process simplifies the assertion so that only the state variable needs to be included in the precondition.

Overlapping Activity

As back-to-back transactions may require helper code, overlapping activity may pose a challenge to model in SVA. An example would be a pipelined protocol where responses return an arbitrary time later. For example, consider a protocol with a transfer start and stop signal combined with a transaction id. A simple assertion would be:

```
assert property ( start | => ##[0:$] stop )
```

Unfortunately, this assertion has mismatches when responses come much later than the requests.

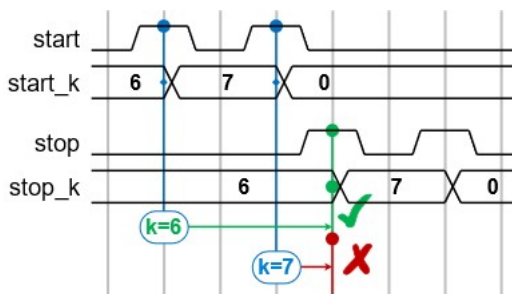


Figure 4 - Overlapping bus protocol activity mismatches.

Figure 4 shows how the property mismatches with transaction 7 because the response for transaction 6 matched with transaction 7. The solution is to use some helper code to keep track of the transaction id so the responses are matched correctly to their requests:

```
bit [2:0] start_k, stop_k;
always @(posedge clk) begin
  if (start)
    start_k <= start_k + 1; // Start counter
  if (stop)
    stop_k <= stop_k + 1; // Stop counter
end
```

This HDL helper code creates a counter for the number of occurrences of *start* and *stop*. The counters need to be wide enough to hold a unique value for each overlapping transaction and the code needs to be synthesizable since it will be used with formal verification. With the counters defined, the transaction id is captured when the transaction starts by grabbing the value of *start_k* using a named property local variable:

```
property overlap_start_stop;
  bit [2:0] k; // New variable for each instance
  (start, k = start_k) // Local variable assignment
  |-> ##[1:4]
  stop && (stop_k == k); // Compare id with local variable
endproperty
assert property overlap_start_stop;
```

When the response occurs (i.e., the *stop* signal), the local variable is compared to the *stop_k* counter to see if it occurs within the response window, allowing the assertion to check the correct response:

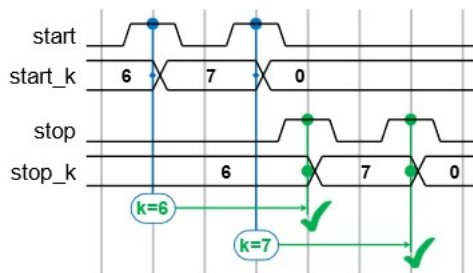


Figure 5 - Solution for overlapping activity.

Obviously, out-of-order responses would call for a more sophisticated solution than shown here.

Constraining Inputs

Another scenario for using helper code is when defining formal input constraints. A good example is a start of packet and end of packet protocol. Writing constraints that are flexible enough to allow for multiple packets, match up to the end of packet with the start of packet, and stay valid long enough for all the data to be transferred is not as straightforward as it may seem. However, using helper code for a counter and state variables makes constraining the formal tool much easier than writing complex constraints.

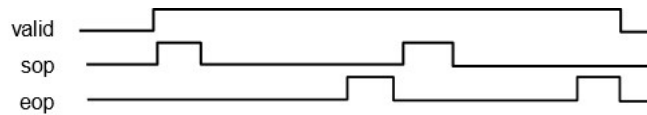


Figure 6 - Packet based bus protocol with start-of-packet (sop) and end-of-packet (eop).

The first step is to define an active flag to indicate that the packet is being transferred once the start of packet occurs:

```
logic pkt_started; // Indicates packet started
logic active; // Packet being transferred
assign active = pkt_sop | pkt_started;
always @(posedge clk or negedge rst_n)
  if ( ~rst_n ) pkt_started <= 1'b0;
  else if ( pkt_eop ) pkt_started <= 1'b0;
  else if ( pkt_sop ) pkt_started <= 1'b1;
```

Since packets typically have variable sizes due to different data payload lengths, an RTL counter helps constrain the length of the packet:

```

always @(posedge clk or negedge rst_n)
    if ( ~rst_n ) count <= 8'h00;
    else if ( pkt_eop ) count <= 8'h00;
    else if ( pkt_valid ) count <= count + BUS_WIDTH;

```

With the counter and active flags defined, writing the SVA properties becomes pretty simple:

```

asm_sop_correct: assume property( pkt_sop |-> ~pkt_started );
asm_idle_noval : assume property( ~active |-> ~pkt_val );
asm_active_val : assume property( active |-> pkt_val );
asm_idle_nosop : assume property( ~active |-> ~pkt_sop );
asm_idle_noeop : assume property( ~active |-> ~pkt_eop );
asm_noval_nosop: assume property( ~pkt_val |-> ~pkt_sop );
asm_noval_noeop: assume property( ~pkt_val |-> ~pkt_eop );
asm_transaction: assume property( pkt_sop |-> pkt_val[*1:$] ##0
                                pkt_eop );
asm_eop_correct: assume property( pkt_eop == (count>=length));

```

These constraints can be further reduced using the equivalence operator (<=>):

```

asm_pkt_vld      : assume property ( active <=> pkt_vld );
asm_pkt_sop      : assume property ( $rose(pkt_vld) |-> $rose(pkt_sop));
asm_pkt_eop      : assume property ( pkt_vld && (count>=length) <=>
                                pkt_eop );
asm_pkt_notsop   : assume property ( pkt_started |-> !pkt_sop );

```

In these constraints, the *active*, *pkt_started*, and *count* variable helper code generate the start of packet and end of packet signals. The *count* restricts the length of the packet, the length of the packet constrains the end of the packet, the end of packet constrains the valid packet signal, and so on. With the aid of helper code, the formal constraints are reduced to only four assumptions.

Design Modeling

An obvious area for helper code is modeling design behavior for testing purposes. For example, ECC logic might be modeled as:

```

// ECC implementation
wire logic p1 = 1 ^ data[0] ^ data[1] ^ data[3] ^ data[4] ^ data[6];
wire logic p2 = 1 ^ data[0] ^ data[2] ^ data[3] ^ data[5] ^ data[6];
wire logic p4 = 1 ^ data[1] ^ data[2] ^ data[3] ^ data[7];

wire logic p8 = 1 ^ data[4] ^ data[5] ^ data[6] ^ data[7];

assign expected_encoded_data[11:0] = {
    data[7], data[6], data[5], data[4], p8, data[3],
    data[2], data[1], p4, data[0], p2, p1 };

```

While the ECC logic could be written inside the property, it would not be as easily understood as breaking up the terms of the calculation into simple Verilog code. This modeling code makes it easy to understand the ECC algorithm being used to check the design's output. If described in an SVA property, the modeling logic would be large and harder to follow.

```

// Checker
assert property ( encoded_data == expected_encoded_data );

```

Using Generates

Of course, some SystemVerilog features cannot be included in a property. An example of that is using a generate statement. Generate statements are useful for configuring your formal IP into a checker, a coverage collector, or input constraints. Using parameters, we can selectively decide to instantiate our asserts, covers, or assumptions.

```

module assertions
  #(parameter ASSERTS = 1,    // enable assertion properties
    COVERS = 1,              // enable cover properties
    ASSUMES = 1,             // enable assumption properties
    ... )
  ( /* port list */ );
  ...
  generate
    if (ASSERTS==1) begin: gen_asserts
      assert property ...
    end
    if (COVERS==1) begin : gen_covers
      cover property ...
    end
    if (ASSUMES==1) begin : gen_assumes
      assume property ...
    end
  endgenerate
endmodule

```

Generates also help with applying parameterized properties across a number of consecutive elements of an array or structure. In this example of an Ethernet packet, the *prop_pkt()* property is used to initialize each element of a packet array. The argument passed into the property by the generate statement selects the element of the packet array that gets initialized:

```

property prop_pkt( n );
  if ( packet[n].kind == MAC_ARP ) prop_pkt_data(n+1)
  else if ( packet[n].kind == MAC_IPV4 ) prop_pkt_ipv4(n+1)
  ...
endproperty

for (genvar i = 0; i < NUM_ITEMS-1; i++)
begin: pkt_init
  assume property( pkt_valid |-> prop_pkt(i) );
end

```

Each individual assumption could have been explicitly written, but using a *generate for* loop simplifies and automates the writing of the constraints and avoids unintentional typos when instantiating manually (see [1] for a working example).

Simplifying Liveness to Safety

Helper code can also help re-write our SVA properties. In fact, there are two ways to structure a property—as a *liveness property* or as a *safety property*. With a liveness property, the question is asked, “Is something alive?” For example, when a request arrives, is there eventually an acknowledgement? Formal knows how to handle liveness properties, but liveness properties can be much harder to solve than safety properties.

While liveness properties look for something good to happen, safety properties check that nothing bad happens. In general, most of our properties and assertions are safety properties, asserting that bad behavior does not occur. With safety properties, formal has only to find a counter-example, which may be quite simple; whereas, liveness properties may require a full proof or deep state space analysis. So if a liveness property can be written as a safety property, formal analysis may run faster and converge more quickly on a proof.

One way to re-write a liveness property is with the help of Verilog helper code. Consider the following example:

```

ast_liveness: assert property ( a |-> strong( ##[0:$] b ) );

```

The liveness property is waiting for *b* to happen once *a* occurs. Using an always process and creating a state variable, *a* can be tracked as follows:

```

logic past_a;
always @(posedge clk or negedge resetn)
  if ( !resetn )
    past_a <= 0;
  else
    past_a <= past_a | a;

```

With this helper code, the liveness property is simplified to the following:

```

ast_safety: assert property ( b |-> past_a );

```

Formal will try to cause *b* to assert, and when it does, it will look to see if *a* was observed by the always process and *past_a* was set. Formal only needs to look for a counter-example versus trying to prove the liveness property for all time, which may improve formal performance and help the property to converge on a solution.

Random Variables

A very common use for helper code is for generating formal controlled variables known as *free variables*. Free variables are essentially *random* because formal can set them to any value if left unconstrained. These free or unconstrained variables are also known as *non-deterministic variables*. A common usage for non-deterministic variables is generating random data values.

For example, suppose we want to put random data on our output. With formal, we simply define the variable without a driver, and then write a constraint to set the output equal to our random variable.

```
module assertions( ... output_data );
  byte data;           // Data output
  byte random_data;    // Unconstrained data
  ...

  always @(posedge clk or negedge rst_n)
    if ( !rst_n )
      data <= '0;
    else begin
      data <= random_data;
      assume property ( output_data == data );
    end
endmodule
```

Of course, this does not specify what should happen during reset. We can further constrain this with some more helper code:

```
module assertions( ... output_data );
  byte random_data;    // Unconstrained data
  ...
  assume property ( output_data == random_data );
endmodule
```

Now the helper code handles resetting the data output, but this helper code could be much more sophisticated to handle other scenarios or drive the random data onto the output at specific times.

Another good use of a formal random variable is as a condition in our formal targets. Properties targeting a specific scenario and using a random variable force formal to assign the value to the formally controlled variable to hit the target.

For example, consider testing a safety mechanism using a sequential logic equivalency checking (SLEC) formal engine to perform the formal analysis. By comparing the original RTL design with a faulted model, it can be determined if the safety mechanism detects the error by checking the two safety mechanism's error signal (see Figure 7). If the injected fault is detected, then the safety mechanism of the faulted version should signal the error while the non-faulted model indicates no error detected.

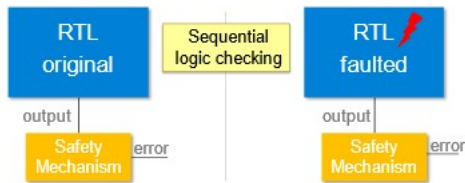


Figure 7 - Testing a safety mechanism with an injected fault.

To inject the fault, we use a formal directive to conditionally cut the signal of interest in the *faulted* model and conditionally toggle its value when the formal controlled *fault* signal equals a specific value:

```
// Inject fault
netlist cutpoint faulted.signal -cond {fault==1} -driver \
~faulted.signal
```

Using some helper Verilog code, a couple of flags are set to indicate a safety violation has occurred and whether the safety mechanism has detected it:

```
int fault;
always @($global_clock) begin
  violation = injected && ( original.output != faulted.output );
  detected = injected && ( !original.error && faulted.error );
  ...
end
```

The formal target, a cover property, specifies the random variable *fault* to the same value as used in the conditional cut-point:

```
// Find residual fault(s)
cover property (( fault == 1 ) && violation && !detected );
```

For formal to reach this cover property, it sets the formally controlled signal *fault* to 1, which injects the fault into the design and then determines if the safety goal was violated and/or detected (see [2] for a detailed description of this approach).

Here again, we see that without using helper code we could not have accomplished our safety formal analysis. The random variable helps us direct formal to prove the specific fault that we are interested in testing, and it is also used by the formal directive to automatically inject the fault.

Abstractions

Another common application for helper code is with abstraction. Abstraction helps formal analysis to remove design and state space complexity so formal can converge on a solution more easily.

One design construct that often has a large state space is a counter, and counters may be used in many places like arbiters that use multiple counters for implementing a credit scheme. When we find a way to abstract the counters, the state space may be significantly reduced, allowing formal to focus on verifying the design instead of traversing through the counter's state space.

Our abstraction needs to provide the correct initialization of the counter and implement the correct functionality, including the counter wrapping. Since the counter is not the target of our formal analysis, we assume that the counter's design implementation is correct. To illustrate, consider a basic counter:

```
always @(posedge reset or posedge clock)
  if ( reset )
    count <= 0;
  else if ( count == MAX )
    count <= 0;
  else
    count <= count + 1;
```

Abstracting the counter simply involves making the counter a formal control point using a formal directive:

```
netlist cutpoint count
```

Next, a constraint defines the counter's functionality using a formal control point or free variable:

```
logic [3:0] increment;
assume property ( count == ($past(count) + increment) % MAX );
```

The *increment* free variable allows formal to choose any value of the counter and the wrapping is handled by modulus with MAX. Since formal can pick any value, it can jump to any required state space to converge quickly on properties that have the counter in their cone-of-influence.

With Questa Formal, the same abstraction can be accomplished using just one formal directive:

```
formal abstract counter
```

This command automatically abstracts all counters that Questa Formal detects. Therefore, counter abstractions using Questa Formal are straightforward using various helper code of tool directives and/or a Verilog code.

Another example of using a formal control point for an abstraction is an arbiter that consists of *design symmetry*. A basic example is a round robin arbiter:


```

always @* begin
  case (pointer_reg)
    2'b00 :
      if (req[0]) grant = 4'b0001;
      else if (req[1]) grant = 4'b0010;
      else if (req[2]) grant = 4'b0100;
      else if (req[3]) grant = 4'b1000;
      else grant = 4'b0000;
    2'b01 :
      if (req[1]) grant = 4'b0010;
      else if (req[2]) grant = 4'b0100;
      else if (req[3]) grant = 4'b1000;
      else if (req[0]) grant = 4'b0001;
      else grant = 4'b0000;
    2'b10 :
      if (req[2]) grant = 4'b0100;
      else if (req[3]) grant = 4'b1000;
      else if (req[0]) grant = 4'b0001;
      else if (req[1]) grant = 4'b0010;
      else grant = 4'b0000;
    2'b11 :
      if (req[3]) grant = 4'b1000;
      else if (req[0]) grant = 4'b0001;
      else if (req[1]) grant = 4'b0010;
      else if (req[2]) grant = 4'b0100;
      else grant = 4'b0000;
  endcase
end

```

The arbiter has an implied priority specified by the *if* statements. A simple circular pointer is used for the round robin selection:

```

logic [1:0] pointer_reg;

always @(posedge clk or posedge reset)
  if ( reset )
    pointer_reg <= '0;
  else
    pointer_reg <= pointer_reg + 2'b1;

```

Since the design is symmetrical (*i.e.*, requiring no special cases to be modeled), it is sufficient to convert the arbiter pointer into a formal control point to abstract it.

Using formal directives as the helper code, the *pointer_reg* is cut so it becomes a formal control point:

```

netlist cutpoint pointer_reg

```

Now the arbiter can jump to any state needed to complete its formal proof. Likewise, the *grant* output could be turned into a formal control point. In that case, the arbiter itself becomes a black box, allowing formal to drive any *grant* values it wishes (Figure 8).

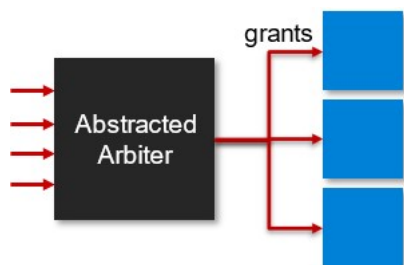


Figure 8 - With the output cut, the arbiter becomes an abstracted black box.

In this case, the *grant* will need constrained so formal only assigns one grant at a time. Furthermore, additional constraints are needed to model the round robin behavior of the arbiter for formal analysis. For that reason, the first solution of turning the round-robin pointer into a formal control point is better since the RTL handles constraining the formal tool to the round robin behavior.

Formal Directives

Throughout these examples, we have shown that both Verilog code and formal directives can serve as helper code for our properties and formal analysis. Formal directives are useful for initializations, creating incremental properties, defining constants and constrains that reduce the state space, abstracting design constructs like counters, memories, and FIFOs, SLEC mappings to define equivalency points, automating our formal flow, interactive formal for bug hunting, and various other formal techniques like goal posting [3].

Formal tools provide formal directives in the form of Tcl commands. QuestaFormal provides an extensive Tcl interface for automating formal analysis. Moreover, QuestaFormal has TCL APIs, allowing traversal of formal results from various apps and even the design’s netlist. The following table shows a number of applications to use with QuestaFormal’s APIs:

QuestaFormal APIs	Example Applications
Autocheck	Controlling apps
XCheck	Traversing or examining results
Covercheck + CoverGroupcheck	Creating automatic properties
Propcheck	Automatic abstractions
Sleccheck	Automatic SLEC mappings
CDC (plus 3 other CDC APIs)	Analyzing a design
Resetcheck + ResetStaticcheck	Analyzing synchronizers
ClockTree	Fault injection
ResetTree	Reporting on results
Directives	Generating coverage exclusions
Netlist	Goal posting [3]

As mentioned in the previous section, a simple formal directive can abstract all the counters in a design:

```
formal abstract counter
```

However, suppose only the counters in a particular cone of influence need abstracted. Using Questa Formal’s cone-of-influence (COI) and netlist APIs, the design can be traversed to cut only those counters found in the cone of a specific signal like a design’s output signal

```

# Download at: doulos.com/TclAbstractCounter
set record [formal generate coi -var $signal]
foreach elem [COI::GetCounters $record] {
    # Skip any assumption signals/nets
    if { [regexp -nocase assume [COI::GetCategory $elem]] } { continue }
    set signal [COI::GetName $elem]
    if { [catch { set net [Netlist::GetNet $signal] } msg ] } {
        mti_message "Skipping $net $signal ...\n"
        continue
    }
    if { [catch { set width [Netlist::GetNetWidth $net] } msg ] } {
        mti_message "Skipping $net $signal ...\n"
        continue
    }
    if { [regexp {<(\S+)>} $net -> indices] } {
        if { ![regexp {(\d+):(\d+)} $indices -> msb lsb] } {
            set msb $indices
            set lsb $indices
        }
    } else {
        set msb [expr $width-1]
        set lsb 0
    }
    regsub {<.*>} $net {} net
    for {set i $lsb} {$i <= $msb} {incr i} {
        set netSlice "$net<$i>"
        if { ![catch { set name [Netlist::GetName $netSlice] } msg ] } {
            regsub {[^.]+"} $signal $name netName
            mti_message "Adding counter cutpoint for signal $netName ...\n"
            netlist cutpoint $netName
        }
    }
    mti_message "\n"
}

```

In this example, the COI Tcl API returns all the design elements of a signal's COI and the `COI::GetCounters()` call returns any counters detected in the COI. Then using the Netlist API, each bit of the counter vector is traversed and the `netlist cutpoint` command is used to cut the counter's drivers. A useful case for this script is for formal targets struggling to converge due to counters in the property's COI. Care must be taken when abstracting any design element(s) so formal does not return false results, but if the proof is still valid with an abstraction then this script provides a simple way to search and remove them.

To the Rescue

Most design behaviors are easily described using SVA syntax, but for everything else helper code comes to the rescue. Helper code comes in the form of Verilog processes, undriven random variables, generate statements, tasks and functions, and formal directives that live outside of our SystemVerilog code. Helper code can significantly simplify writing properties and modeling complex behaviors. It is crucial for creating abstractions and adding formal directives to help with formal analysis, and it enables automation of the formal process. If you are looking to boost your formal verification productivity, consider adding helper code into your next formal project.

References

1. Doug Smith. "Doing the Impossible: Using Formal Verification on Packet Based Data Paths." DVCon San Jose, February 2023.
2. Ibid. "Formal Flow for Automotive Safety: Bulletproofing Car Design (<https://circuitcellar.com/research-design-hub/formal-flow-for-automotive-safety/>)." Circuit Cellar Magazine, Issue 344 (<https://circuitcellar.com/inside-circuit-cellar-2019/>), March 2019.
3. Richard Ho, et al., "Post-Silicon Debug Using Formal Verification Waypoints," DVCon 2009.

[Back to Top](#)

[Table of Contents](#)

Verification Horizons Articles:

- + [Everything, everywhere, all at once: Big data reimagines verification predictability and efficiency \(https://verificationacademy.com/verification-horizons/march-2023-volume-19-issue-1/big-data-reimagines-verification-predictability-and-efficiency\)](https://verificationacademy.com/verification-horizons/march-2023-volume-19-issue-1/big-data-reimagines-verification-predictability-and-efficiency)
- + [Democratizing digital-centric mixed-signal verification methodologies \(https://verificationacademy.com/verification-horizons/march-2023-volume-19-issue-1/democratizing-digital-centric-mixed-signal-verification-methodologies\)](https://verificationacademy.com/verification-horizons/march-2023-volume-19-issue-1/democratizing-digital-centric-mixed-signal-verification-methodologies)
- + [Lane margining at receiver and its application through pipe message bus \(https://verificationacademy.com/verification-horizons/march-2023-volume-19-issue-1/lane-margining-at-receiver-and-its-application-through-pipe-message-bus\)](https://verificationacademy.com/verification-horizons/march-2023-volume-19-issue-1/lane-margining-at-receiver-and-its-application-through-pipe-message-bus)
- + [The RISC-V Verification Interface \(RVVI\) – test infrastructure and methodology guidelines \(https://verificationacademy.com/verification-horizons/march-2023-volume-19-issue-1/risc-v-verification-interface-rvvi-test-infrastructure-and-methodology-guidelines\)](https://verificationacademy.com/verification-horizons/march-2023-volume-19-issue-1/risc-v-verification-interface-rvvi-test-infrastructure-and-methodology-guidelines)
- + [A formal-based approach for efficient RISC-V processor verification \(https://verificationacademy.com/verification-horizons/march-2023-volume-19-issue-1/a-formal-based-approach-for-efficient-risc-v-processor-verification\)](https://verificationacademy.com/verification-horizons/march-2023-volume-19-issue-1/a-formal-based-approach-for-efficient-risc-v-processor-verification)

- 1/formal-based-approach-for-efficient-risc-v-processor-verification)
- + Jumpstart your formal verification with a little help
- + Resolving metastability issues for multi-clock SOC environment for I2C (<https://verificationacademy.com/verification-horizons/march-2023-volume-19-issue-1/resolving-metastability-issues-for-multi-clock-soc-environment-for-i2c>)

SIEMENS

Siemens Digital Industries Software

#TodayMeetsTomorrow
(<https://twitter.com/search?q=%23todaymeetstomorrow>)



(<https://www.facebook.com/siemensdigitalindustriessoftware/>)

Portfolio

Cloud

(<https://www.sw.siemens.com/en-us/cloud/>)

US/digital-

transformation/cloud/)

Mendix

(<https://www.mendix.com/>)

Electronic Design

Automation

(<https://eda.sw.siemens.com/en-us/>)

MindSphere

(<https://www.plm.automation.siemens.com/global/en/products/mindsphere/>)

Design, Manufacturing

and PLM Software

(<https://www.plm.automation.siemens.com/global/en/story/customers/>)

View all Portfolio

(<https://www.sw.siemens.com/en-us/portfolio/>)

Explore

Community

(<https://community.sw.siemens.com/>)

Blog

(<https://blogs.sw.siemens.com/>)

Online Store

(<https://www.dex.siemens.com/us/>)

Siemens

About Us

(<https://www.sw.siemens.com/en-us/>)

US/)

Careers

(<https://www.sw.siemens.com/en-us/hr>)

US/)

Events

(<https://www.plm.automation.siemens.com/global/en/yc>)

success/events/)

News and Press

(<https://www.plm.automation.siemens.com/global/en/ou>)

story/mindsphere/)

Customer Stories

(<https://support.sw.siemens.com/>)

(<https://www.plm.automation.siemens.com/global/en/ou>)

Partners

(<https://www.plm.automation.siemens.com/global/en/ou>)

story/partners/)

Trust Center

(<https://www.sw.siemens.com/en-us/trust-center>)

Contact

VA - Contact Us

(<https://verificationacademy.com/>)

PLM - Contact Us

(<https://www.plm.automation.siemens.com/>)

us/hr)

EDA - Contact Us

(<https://resources.sw.siemens.com/>)

US/)

Worldwide Offices

(<https://www.plm.automation.siemens.com/global/en/ou>)

story/offices.html)

Support Center

(<https://support.sw.siemens.com/>)

US/)

Give us Feedback

(<https://webtac.industrysoft.com/>)

